20. SOLID Prensipleri AI Promptlarına Nasıl Çevrilir?

Yapay Zekanın Spagetti Kod Üretmesini Engelleyen 5 Altın Mimari Kural
Bölüm 4 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"SOLID prensipleri 20 yıl önce nesne yönelimli programlama için çıktı. Peki 2026'da LLM'lere kod yazdırırken SOLID neden hâlâ en büyük kurtarıcımız?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 AI modelleri serbest bırakıldığında her işi tek bir fonksiyona yığma (God Function) eğiliminde-dir. - 📌 SOLID prensiplerini prompt kısıtı olarak vermek (Örn: 'Single Responsibility uygula'), AI'ın modüler kod üretmesini sağlar. - 📌 Bağımlılıkların Tersine Çevrilmesi (Dependency Inversion), test edilebilir ve mocklanabilir kodun anahtarıdır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

İsviçre çakısını elinize alın: Üzerinde bıçak, makas, tirbüşon, testere ve tırnak törpüsü var. Küçük bir piknik için harikadır; ancak bir ameliyathaneye girdiğinizde cerrahın elinde İsviçre çakısı görürseniz masadan kaçarsınız!

Cerrahın elinde sadece tek bir işi mükemmel yapan neşter olmalıdır.

Yapay zekaya *"Kullanıcı kaydı yap"* dediğinizde size tam bir İsviçre çakısı üretir: Aynı fonksiyon hem şifreyi doğrular, hem veritabanına yazar, hem e-posta atar, hem PDF fatura üretir! Kod 300 satır olur ve ilk değişiklikte patlar.

SOLID prensiplerini prompt'a eklediğinizde ise yapay zeka neşterler üretir: Her sınıf ve fonksiyon tek bir işi kusursuz yapar.

🧠 2. AI Çağında Yeni Zihniyet Modeli

SOLID Prensiplerinin Prompt Karşılıkları:

```
S – Single Responsibility: "Bu fonksiyon sadece tek bir iş yapmalı. E-posta
gönderimini ayrı bir servise ayır."
O – Open/Closed: "Yeni bir ödeme yöntemi eklendiğinde mevcut koda dokunulmamalı; bir
interface implemente edilmeli."
L – Liskov Substitution: "Alt sınıflar üst sınıfın davranışını bozmamalı,
beklenmedik hata fırlatmamalı."
I – Interface Segregation: "Müşteriyi kullanmadığı metotları içermeye zorlayan
devasa interfaceler yazma; küçük arayüzler kur."
D – Dependency Inversion: "Doğrudan somut sınıfa (Concrete Class) değil, bir
Interface/Abstraksiyona bağımlı ol."
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: SOLID ve Temiz Mimari Uzmanı
Aşağıdaki 'God Object' (her işi yapan) JavaScript fonksiyonunu SOLID prensiplerine
göre refactor et.

## MEVCUT KOD:
```typescript
class OrderProcessor {
  async process(order: any) {
    // 1. Doğrulama
    if (order.amount <= 0) throw new Error("Hatalı tutar");
    // 2. SQL kaydı
    await db.query(`INSERT INTO orders VALUES (${order.id}, ${order.amount})`);
    // 3. E-posta bildirimi
    await mailer.send(order.email, "Sipariş alındı");
  }
}
````

GÖREV:

1. Single Responsibility (SRP) uyarınca doğrulama, veritabanı ve bildirim işlerini ayrı servislere böl.
2. Dependency Inversion (DIP) uyarınca `OrderProcessor` sınıfına bağımlılıkları constructor üzerinden interface olarak enjekte et.
````

````
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// SOLID'e Aykırı: Her işi tek sınıfta yapan God Object
export class OrderProcessorNaive {
  async processOrder(order: any) {
    // SRP ihlali: Validasyon, DB erişimi ve Mail tek yerde
    if (!order.id || order.amount <= 0) throw new Error("Geçersiz");
    // SQL Injection açığı da var!
    await db.query(`INSERT INTO orders VALUES ('${order.id}', ${order.amount})`);
    // DIP ihlali: Dış kütüphaneye doğrudan sıkı bağımlılık (tight coupling)
    await fetch("https://api.sendgrid.com/send", { body: JSON.stringify(order) });
  }
}
````
````

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// SOLID Uyumlu: Modüler, Test Edilebilir, Tip Güvenli Mimari

// 1. Abstraksiyonlar (DIP)
export interface OrderRepository {
  save(order: ValidatedOrder): Promise<void>;
}

export interface NotificationService {
  notifyOrderPlaced(order: ValidatedOrder): Promise<void>;
}

export interface ValidatedOrder {
  readonly id: string;
  readonly amount: number;
  readonly customerEmail: string;
}

// 2. Orkestrasyon Sınıfı (SRP: Sadece iş akışını yönetir)
export class OrderProcessor {
  constructor(
    private readonly repository: OrderRepository,
    private readonly notifier: NotificationService
  ) {}

  async process(order: ValidatedOrder): Promise<void> {
    // Adım 1: Kaydet
    await this.repository.save(order);

    // Adım 2: Bildir
    await this.notifier.notifyOrderPlaced(order);
  }
}
```

💡 **Kod Analizi:** Artık `OrderProcessor` veritabanının PostgreSQL mi MongoDB mi olduğunu veya e-postanın SendGrid ile mi AWS SES ile mi gittiğini bilmez. Test yazarken sahte (mock) repository enjekte etmek 2 satırdır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** SOLID prensiplerini aşırıya kaçırıp 3 satırlık kod için 15 farklı interface ve dosya üretmek (Over-engineering).
  - **Çözüm:** Pragmatik olun; SOLID prensiplerini sistemin büyüyeceği ve değişeceği kritik noktalara uygulayın.
- **Tuzak:** AI'a sadece 'SOLID uygula' deyip hangi prensibe odaklanacağını belirtmemek.
  - **Çözüm:** Özellikle 'SRP ve Dependency Injection uygula' şeklinde spesifik kurallar verin.
- **Tuzak:** Interface yazıp constructor yerine sınıf içinde `new ConcreteClass()` çağırmak.

- **Çözüm:** Bağımlılıkları daima dışarıdan constructor parametresi olarak geçin (Inversion of Control).

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 20: SOLID Prensipleri - AI Kodunu Mimari Seviyeye Taşımak*

- 🖥️ **Slayt 1:** Neden yapay zeka varsayılan olarak 'God Object' üretir?
- 🖥️ **Slayt 2:** 5 SOLID prensibinin sadeleştirilmiş Türkçe açıklaması.
- 🖥️ **Slayt 3:** Dependency Inversion ile test edilebilir kod üretmek.
- 🖥️ **Slayt 4:** Canlı Refactoring: 150 satırlık dev fonksiyonu SOLID ile parçalamak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Tek bir sınıf içinde hem veritabanına yazan hem e-posta atan fonksiyonu AI'a verin. 'Apply Single Responsibility and Dependency Inversion' promptuyla iki ayrı interface ve sınıf oluşturmasını sağlayın. Ardından mock kullanarak Jest testi yazdırın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Kod tabanınızda 100 satırdan uzun olan bir fonksiyon bulun. İçindeki işleri listeleyin ve en az bir işi (örn: e-posta veya loglama) ayrı bir servise çıkartın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: SOLID'in 'Single Responsibility Principle' (Tek Sorumluluk Prensibi) kuralı neyi ifade eder? - A) Şirkette sadece bir yazılımcının çalışmasını - B) Bir sınıfın veya fonksiyonun değişmesi için yalnızca tek bir nedeninin olması gerektiğini - C) Sadece tek bir satır kod yazılmasını - D) Değişkenlerin tek harfle isimlendirilmesini > **Doğru Yanıt:** B > **Açıklama:** SRP, bir modülün sadece tek bir aktöre/iş kuralına karşı sorumlu olması gerektiğini; farklı sorumlulukların birbirine karıştırılmamasını savunur.

Soru 2: Dependency Inversion (Bağımlılıkların Tersine Çevrilmesi) prensibinin en büyük pratik avantajı nedir? - A) Kodun daha yavaş çalışması - B) Sınıfların somut veritabanlarına sıkı sıkıya bağlanmasını engelleyerek birim testlerde (Unit Test) kolayca mocklanabilmesini sağlaması - C) Sadece Java dilinde çalışması - D) CSS hatalarını düzeltmesi > **Doğru Yanıt:** B > **Açıklama:** Interface bağımlılıkları sayesinde test koşarken gerçek veritabanı yerine sahte (mock) objeler verilebilir; testler izole ve yıldırım hızında çalışır.